

05 · TF-IDF

TF-IDF combina:

- **TF** (*Term Frequency*): $TF(t, d) = f(t, d) / N(d)$ — proporción del documento que ocupa el término.
- **IDF** (*Inverse Document Frequency*): $IDF(t) = \log(N/n(t))$ — cuán raro es el término en el corpus.
- **TF-IDF** = $TF \times IDF$: alto cuando el término es frecuente en ese documento y raro en los demás → **término distintivo**.

Mismo corpus que Bag of Words: las **descripciones de producto** (`datos/productos.csv`).

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")

def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
import re
import unicodedata

import nltk
import spacy

nlp = spacy.load("es_core_news_sm")
STOPWORDS = set(nltk.corpus.stopwords.words("spanish"))

def _limpiar(texto):
    # NFKC junta tildes combinantes sueltas; luego minúsculas y solo letras/espacios
    texto = unicodedata.normalize("NFKC", str(texto)).lower()
    return re.sub(r"^[^\w\s]", " ", texto)

def _lemas(doc):
    return [
        t.lemma_.lower()
```

```

        for t in doc
            if t.is_alpha and not t.is_stop and t.lemma_.lower() not in STOPWORDS and
len(t.lemma_) > 2
        ]

def normalizar(texto):
    """minúsculas -> sin signos -> sin stopwords -> lematizado. Devuelve un str."""
    return " ".join(_lemas(nlp(_limpiar(texto))))

def normalizar_muchos(textos):
    """Igual que normalizar() pero en lote con nlp.pipe (más rápido para un
corpus)."""
    return [" ".join(_lemas(doc)) for doc in nlp.pipe([_limpiar(t) for t in textos],
batch_size=64)]

```

```

productos = cargar("productos.csv")
docs = productos["descripcion"].fillna("").tolist()
docs_norm = normalizar_muchos(docs)
print(f"{len(docs_norm)} documentos normalizados")

```

Leyendo ../datos/productos.csv (27.5 KB)
90 documentos normalizados

IDF a mano: verificar la intuición

$IDF(t) = \log(N/n(t))$. Un término que aparece en casi todas las descripciones (pantalla) tiene IDF bajo; uno raro (titanio) IDF alto; y uno que aparece en **todos** los documentos tiene IDF = $\log(1) = 0$.

```

import numpy as np

docs_norm_demo = [d + " marcavisible" for d in docs_norm] # término presente en
TODOS
N = len(docs_norm_demo)
for termino in ["pantalla", "camara", "bateria", "titanio", "hdr", "marcavisible"]:
    n_t = sum(1 for d in docs_norm_demo if termino in d.split())
    idf = np.log(N / n_t) if n_t else float("nan")
    print(f" {termino:13} aparece en {n_t:2}/{N} docs    IDF = {idf:.3f}")

```

pantalla	aparece en 35/90 docs	IDF = 0.944
camara	aparece en 13/90 docs	IDF = 1.935
bateria	aparece en 15/90 docs	IDF = 1.792
titanio	aparece en 3/90 docs	IDF = 3.401
hdr	aparece en 2/90 docs	IDF = 3.807
marcavisible	aparece en 90/90 docs	IDF = 0.000

En **scikit-learn** el IDF lleva un +1 de suavizado ($idf = \ln((1+N)/(1+n(t))) + 1$), así que un término omnipresente **no llega a 0** exactamente, pero sí obtiene el peso más bajo y casi constante de toda la matriz.

TF-IDF con scikit-learn

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(min_df=2)
X = vectorizer.fit_transform(docs_norm)
vocab = vectorizer.get_feature_names_out()
print(f"Matriz TF-IDF: {X.shape[0]} x {X.shape[1]}")

df_tfidf = pd.DataFrame(X.toarray(), columns=vocab, index=productos["codigo"])
df_tfidf.iloc[:5, :12]
```

Matriz TF-IDF: 90 x 253

	abatible	acceso	accion	actividad	adaptativo	adicional	agua	ajustar	alexa	alma
codigo										
PROD-001	0.0	0.0	0.0	0.0	0.0	0.0	0.170191	0.0	0.0	
PROD-002	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	
PROD-003	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	
PROD-004	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	
PROD-005	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	

Términos más distintivos de cada producto

```
for i in [0, 2, 20, 45]:
    fila = df_tfidf.iloc[i].sort_values(ascending=False).head(6)
    print(f"\n[{productos.iloc[i]['codigo']}] {productos.iloc[i]['nombre']}")
    print(" " + ", ".join(f"{w} ({v:.2f})" for w, v in fila.items()))
```

[PROD-001] Apple iPhone 15 Pro Max 256GB
island (0.27), teleobjetivo (0.27), triple (0.27), pro (0.25), premium (0.25),
dynamic (0.25)

[PROD-003] Samsung Galaxy S24 Ultra 512GB
optico (0.31), dynamic (0.31), incorporo (0.31), titanio (0.31), marco (0.30),
artificial (0.30)

[PROD-021] Sony WH-1000XM5
hora (0.37), dedicado (0.29), over (0.29), minuto (0.29), microfono (0.28),
inalambrico (0.26)

[PROD-046] Logitech G502 Hero
perfil (0.42), dpi (0.39), mouse (0.39), juego (0.37), gamer (0.33), sensor (0.33)

Común vs distintivo: BoW vs TF-IDF

El mismo término pesa poco en TF-IDF si es común en el corpus y mucho si es raro, aunque su conteo bruto (BoW) sea alto.

```

from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(min_df=2).fit(docs_norm)
conteo = cv.transform(docs_norm)
idx = {w: k for k, w in enumerate(cv.get_feature_names_out())}

filas = []
for termino in ["pantalla", "camara", "titanio", "bluetooth", "periscopico"]:
    if termino in idx and termino in vocab:
        docs_con = int((df_tfidf[termino] > 0).sum())
        filas.append({
            "termino": termino,
            "docs en que aparece": docs_con,
            "conteo total BoW": int(conteo[:, idx[termino]].sum()),
            "TF-IDF medio (>0)": round(df_tfidf[termino][df_tfidf[termino] >
0].mean(), 3),
        })
pd.DataFrame(filas)

```

	termino	docs en que aparece	conteo total BoW	TF-IDF medio (>0)
0	pantalla	35	38	0.167
1	camara	13	13	0.228
2	titanio	3	3	0.284
3	bluetooth	3	3	0.315

pantalla aparece en muchos documentos: conteo alto pero TF-IDF bajo. **titanio** o **periscopico** aparecen en pocos: aunque su conteo sea 1, su TF-IDF es alto porque son **distintivos**.

Siguiente: `06_mineria-de-texto.ipynb` — el pipeline completo de principio a fin sobre las reseñas de entrega.